Ψηφιακή Επεξεργασία Εικόνας
Εργασία 3: Image Segmentation

Τζίνα Θεοδώρα (ΑΕΜ: 10715)

Ιούνιος - Ιούλιος 2025

1 Εισαγωγή

2 Εικόνες ως γράφοι

2.1 *image_to_graph.py*

3 Spectral Clustering

3.1 spectral_clustering.*py*

4 Normalized-cuts

4.1 *n_cuts.py*

4.1.1 n_cuts

4.1.2 calculate_n_cut_value

4.1.3 n_cuts_recursive

5 Demo

5.1 demo1.py

5.2 demo2.py

5.3 demo3a.py

5.4 demo3b.py

5.5 demo3c.py

1 Εισαγωγή

Η παρούσα εργασία ασχολείται κατά βάση με το **Image Segmentation**. Αρχικά γίνεται η υλοποίηση μιας βοηθητικής συνάρτησης που μετατρέπει εικόνες σε **γράφους**, υπολογίζει δηλαδή το affinity matrix τους. Έπειτα στο κύριο μέρος της εργασίας υλοποιούνται δύο συναρτήσεις που εκτελούν Image Segmentation. Η πρώτη μέθοδος είναι η **Spectral Clustering** και η δεύτερη η **Normalized-Cuts (n-cuts)**. Η τελευταία εφαρμόζεται σε δύο εκδοχές: αναδρομική και μη-αναδρομική. Με βάση τα δεδομένα που δίνονται πραγματοποιείται η υλοποίηση και εφαρμογή των ανωτέρω μεθόδων, καθώς και η σύγκριση των αποτελεσμάτων τους.

2 Εικόνες ως γράφοι

Η διαδικασία μετατροπής μιας εικόνας σε μορφή "γράφου" αποτελεί το θεμέλιο για την εκτέλεση μεθόδων Image Segmentation. Ουσιαστικά μέσω της διαδικασίας αυτής, κάθε pixel μιας εικόνας αντιστοιχίζεται σε ένα node ή vertex του γράφου, ενώ οι ακμές αυτού αντιπροσωπεύουν την χρωματική ομοιότητα των εκάστοτε pixels. Πιο συγκεκριμένα η συνάρτηση δέχεται έναν πίνακα (M x N x C) **RGB** εικόνας ως είσοδο και επιστρέφει τον πίνακα γειτνίασης (**affinity matrix**) του γράφου. Εφόσον έχουμε μια RGB εικόνα, ο αριθμός των καναλιών C ισούται με 3. Κάθε pixel (π.χ i,j) αντιπροσωπεύει έναν γράφο ενώ το αντίστοιχο βάρος της ακμής του γράφου υπολογίζεται ως εξής:

$$A(i,j) = \frac{1}{e^{d(i,j)}}$$

με d(i,j) η Ευκλείδεια απόσταση της φωτεινότητας των καναλιών μεταξύ του *i*-οστού και του *j*-οστού pixel. Ο γράφος που παράγεται είναι fully-connected, δηλαδή για κάθε ζεύγος κορυφών *i,j* υπάρχει η αντίστοιχη ακμή $e_{i,j}$ με μη-μηδενικό βάρος.

2.1 *image_to_graph.py*

Το αρχείο αυτό εμπεριέχει την ομώνυμη συνάρτηση, η οποία καλείται να υλοποιήσει την μετατροπή εικόνας σε γράφο. Η συνάρτηση δέχεται σαν όρισμα *img_array* μια εικόνα σε μορφή πίνακα, numpy array διαστάσεων (M x N x C), όπου (M x N) είναι οι διαστάσεις της αληθινής εικόνας και C είναι το πλήθος των καναλιών χρώματος. Αρχικά, υπολογίζεται το πλήθος των pixels και αποθηκεύεται σε μια μεταβλητή *num_pixels.* Έπειτα αναδιαμορφώνεται ο αρχικός πίνακας *img_array* από 3D σε 2D, με διαστάσεις (*num_pixels* , C), ώστε να έχει μία γραμμή για κάθε pixel και τρεις στήλες (RGB). Αρχικοποείται ο ζητούμενος τετράγωνος

πίνακας *affinity_mat* (affinity matrix) που ενσαρκώνει τον γράφο και έπειτα για τον υπολογισμό του διατρέχονται μόνο τα άνω τριγωνικά στοιχεία του ανασχηματισμένου πίνακα αφού μιλάμε για έναν συμμετρικό affinity matrix. Μέσω ενός διπλού for loop διατρέχονται όλα τα ζεύγη των pixels και υπολογίζεται μέσω της συνάρτησης numpy.linalg.norm η Ευκλείδεια απόσταση μεταξύ των RGB χρωμάτων. Με την εντολή *weight = np.exp(-dist)* καθορίζεται πόσο "δυνατά" συνδέονται δύο pixels μεταξύ τους, βάσει της χρωματικής τους ομοιότητας, δηλαδή:

- Αν τα pixels έχουν πολύ κοντινά χρώματα, το βάρος είναι κοντά στο 1.
- Αν είναι τελείως διαφορετικά, το βάρος είναι κοντά στο 0.

Τέλος η τιμή *weight,* αποθηκεύεται στην αντίστοιχη θέση του *affinity_mat,* ο οποίος αποτελεί και το αποτέλεσμα της συνάρτησης.

3 Spectral Clustering

Η μέθοδος **Spectral Clustering**, ουσιαστικά αποτελεί τεχνική “φασματικής ομαδοποίησης”. Χρησιμοποιεί έναν affinity matrix, όπου μέσω του φάσματος γίνεται μείωση διαστάσεων πριν από την εφαρμογή μεθόδων ομαδοποίησης σε λιγότερες διαστάσεις. Η γενική προσέγγιση είναι η εφαρμογή μιας πρότυπης μεθόδου ομαδοποίησης (στην περίπτωσή μας είναι η **k-means**) σε κατάλληλα ιδιοδιανύσματα ενός πίνακα **Laplace**. Τα ιδιοδιανύσματα που είναι σχετικά είναι αυτά που αντιστοιχούν στις μικρότερες μη μηδενικές ιδιοτιμές. Η μέθοδος ακολουθεί την εξής διαδικασία:

1. Κατασκευή ενός μη κατευθυντικού γράφου. Έστω **W** ο affinity matrix που περιγράφει τον γράφο.

2. Υπολογισμός του Λαπλασιανού πίνακα **L** ως: **L** = **D** − **W**, όπου **D** ο διαγώνιος πίνακας: $\boldsymbol{D}(i,i) = \sum_i \boldsymbol{W}(i,j)$.

3. Λύση του προβλήματος ιδιοτιμών $\mathbf{Lx} = \lambda\mathbf{x}$, και υπολογισμός των k μικρότερων ιδιοτιμών καθώς και των k ιδιοδιανυσμάτων που αντιστοιχούν σε αυτές τις ιδιοτιμές.

4. Σχηματισμός του πίνακα $\mathbf{U} \in \mathbb{R}^{n \times k}$ που περιέχει τα ιδιοδιανύσματα $\mathbf{u_1}, \ldots, \mathbf{u_k}$ σαν στήλες. Για $i = 1, \ldots, n$, έστω $\mathbf{y_i} \in \mathbb{R}^k$ το διάνυσμα που αντιστοιχεί στην i-οστή γραμμή του **U**.

5. Ομαδοποίηση των σημείων $(\mathbf{y_i})$ $i=1,\ldots,n$ με τον αλγόριθμο k-means στα clusters $C_1, \ldots, C_k$

Η συγκεκριμένη εκδοχή του Spectral Clustering, είναι η πιο απλοποιημένη, μη-κανονικοποιημένη (unormilized) εκδοχή.

3.1 *spectral_clustering.py*

Το συγκεκριμένο αρχείο εμπεριέχει την ομώνυμη συνάρτηση *spectral_clustering* με μεταβλητές εισόδου: *affinity_mat*, έναν numpy array που αντιπροσωπεύει τον γράφο κάποιας εικόνας και *k*, μια μεταβλητή τύπου int που υποδεικνύει το πλήθος των επιθυμητών clusters. Αρχικά υπολογίζεται ο διαγώνιος πίνακας *D*, μέσω της numpy.diag και έπειτα ο πίνακας Laplace *L = D - affinity_mat.*

Για τη λύση του προβλήματος **Lx** = *λ***x**, δηλαδή τον υπολογισμό των ιδιοτιμών και ιδιοδιανυσμάτων του *L*, χρησιμοποιείται η *scipy.sparse.linalg.eigs* ως εξής: ***eigs(L, k=k, which='SR')***, (*'SR'* = Smallest Real για υπολογισμό των *k* ιδιοτιμών/ιδιοδιανυσμάτων με το μικρότερο πραγματικό μέρος). Η συνάρτηση αυτή επιστρέφει δύο πίνακες: *eigenvalues, eigenvectors,* ο πρώτος 1D εμπεριέχει τις ιδιοτιμές και ο δεύτερος 2D εμπεριέχει τα αντίστοιχα ιδιοδιανύσματα. Συγκεκριμένα η κατάταξη αυτών είναι αύξουσα από την/το 1η/ο μέχρι την/το (k-1)η/ο ιδιοτιμή/ιδιοδιάνυσμα.

Έπειτα γίνεται η εισαγωγή των ιδιοδιανυσμάτων σε έναν πίνακα *U* και η κανονικοποίηση αυτού μέσω της *numpy.linalg.norm.* Για την ομαδοποίηση των στοιχείων χρησιμοποιείται η έτοιμη υλοποίηση του αλγορίθμου k-means από το πακέτο *sklearn.cluster.* Καλείται η συνάρτηση *KMeans (random_state = 1* ώστε να επαναλαμβάνεται το πείραμα με το ίδιο random seed και να προκύπτουν ίδια αποτελέσματα ταξινόμησης στα *k* clusters*)* και αφού εφαρμοστεί στον κανονικοποιημένο *U,* μέσω της *kmeans.labels_* λαμβάνεται ο ζητούμενος πίνακας *cluster_idx* τύπου dtype=float και μήκους MN, με τις ετικέτες που δείχνουν σε ποιο cluster ανήκει ο κάθε κόμβος του γράφου εισόδου.

4 Normalized-cuts

Η μέθοδος αυτή διακρίνεται σε 2 εκδοχές (**αναδρομική** και **μη-αναδρομική**) για Image Segmentation και είναι γνωστή ως **Normalized-cuts** ή **n-cuts**.

Για τη **μη-αναδρομική** εκδοχή, τα βήματα υλοποίησης είναι παρόμοια με εκείνα της Spectral Clustering με εξαίρεση το **βήμα 3.** Αφού γίνει ο υπολογισμός των πινάκων **W, D,** και **L**, αναζητείται η λύση του γενικευμένου προβλήματος ιδιοτιμών **Lx** = *λ***Dx**. Υπολογίζονται οι *k* μικρότερες ιδιοτιμές/ιδιοδιανύσματα και ομοίως με πριν γίνεται η ομαδοποίηση στα clusters $C_1, \ldots, C_k$.

Η **αναδρομική** εκδοχή της μεθόδου είναι μια υποπερίπτωση της μη-αναδρομικής για *k* = 2, δηλαδή κάθε φορά χωρίζεται ο γράφος σε **2** κομμάτια. Μετά από κάθε διχοτόμηση (δηλαδή εκτέλεση της μη-αναδρομικής εκδοχής για *k* = 2), ελέγχεται αν θα συνεχιστεί η διχοτόμηση

των συγκεκριμένων κομματιών που προέκυψαν από την διαδικασία ομαδοποίησης (*k*-means). Ο έλεγχος γίνεται ως εξής:

- Αν ο αριθμός των κόμβων είτε με ετικέτα 1 είτε 2 που προκύπτουν είναι μικρότερος από ένα κατώφλι **T1** ή αν η τιμή *Ncut*(*A,B*) είναι μεγαλύτερη από ένα κατώφλι **T2**, τότε η διχοτόμηση σταματά. Η μετρική *Ncut*(*A,B*) για 2 ομάδες κόμβων με ετικέτες "A" και "B" (ή 1 και 0) ορίζεται ως εξής:

$$Ncut(A,B) = 2 - Nassoc(A,B)$$

$$Nassoc(A,B) = \frac{assoc(A,A)}{assoc(A,V)} + \frac{assoc(B,B)}{assoc(B,V)}$$

$$assoc(A,V) = \sum_{u \in A, t \in V} \boldsymbol{W}(u,t)$$

  Ουσιαστικά η μετρική *assoc*(*A,V*) είναι το άθροισμα όλων των βαρών μεταξύ των κόμβων που ανήκουν στην ομάδα A (ή έχουν ετικέτα 1) προς όλους τους κόμβους του γράφου (V). Οι μετρικές *assoc*(*A,A*), *assoc*(*B,V*) και *assoc*(*B,B*) ορίζονται αντίστοιχα. Με **W** συμβολίζεται ο affinity matrix.

- Σε διαφορετική περίπτωση κάθε ένα από τα δύο κομμάτια που προέκυψαν από την διαδικασία ομαδοποίησης χωρίζεται στα 2 ξανά. Η διαδικασία συνεχίζει αναδρομικά και τερματίζει όταν κανένα κομμάτι δεν μπορεί να διχοτομηθεί για τους λόγους που αναφέρθηκαν παραπάνω.

4.1 *n_cuts.py*

Το αρχείο αυτό εμπεριέχει όλες τις απαραίτητες συναρτήσεις υλοποίησης τόσο της μη-αναδρομικής όσο και της αναδρομικής εκδοχής της μεθόδου **Normalized-cuts**.

4.1.1 *n_cuts*

Η συνάρτηση αυτή αποτελεί τον κορμό της μεθόδου και υλοποιεί στην ουσία όλα τα απαραίτητα βήματα της **μη-αναδρομικής** εκδοχής. Δέχεται ως μεταβλητές εισόδου τις *affinity_mat*, έναν numpy array που αντιπροσωπεύει τον γράφο κάποιας εικόνας και *k*, μια μεταβλητή τύπου int που υποδεικνύει το πλήθος των επιθυμητών clusters. Αρχικά ελέγχεται το πλήθος των κόμβων του γράφου πριν ξεκινήσει η διαδικασία διχοτόμησης. Έπειτα, υπολογίζεται ο διαγώνιος πίνακας *D,* μέσω της numpy.diag και στη συνέχεια ο πίνακας Laplace *L = D - affinity_mat.*

Για τη λύση του προβλήματος **Lx** = λ**Dx**, χρησιμοποιείται η *scipy.sparse.linalg.eigs* ως εξής: ***eigs(L, k=k, M=D, which='SR')***, (θυμίζουμε ότι *'SR'* = Smallest Real για υπολογισμό των *k* ιδιοτιμών/ιδιοδιανυσμάτων με το μικρότερο πραγματικό μέρος). Η συνάρτηση αυτή επιστρέφει δύο πίνακες: *eigenvalues, eigenvectors,* ο πρώτος 1D εμπεριέχει τις ιδιοτιμές και ο δεύτερος 2D εμπεριέχει τα αντίστοιχα ιδιοδιανύσματα. Συγκεκριμένα η κατάταξη αυτών είναι αύξουσα από την/το 1η/ο μέχρι την/το (k-1)η/ο ιδιοτιμή/ιδιοδιάνυσμα.

Έπειτα γίνεται η εισαγωγή των ιδιοδιανυσμάτων σε έναν πίνακα *U* και η κανονικοποίηση αυτού μέσω της *numpy.linalg.norm.* Για την ομαδοποίηση των στοιχείων χρησιμοποιείται η έτοιμη υλοποίηση του αλγορίθμου k-means από το πακέτο *sklearn.cluster.* Καλείται η συνάρτηση *KMeans (random_state = 1* ώστε να επαναλαμβάνεται το πείραμα με το ίδιο random seed και να προκύπτουν ίδια αποτελέσματα ταξινόμησης στα *k* clusters*)* και αφού εφαρμοστεί στον κανονικοποιημένο *U,* μέσω της *kmeans.labels_* λαμβάνεται ο ζητούμενος πίνακας *cluster_idx* τύπου dtype=float και μήκους MN, με τις ετικέτες που δείχνουν σε ποιο cluster ανήκει ο κάθε κόμβος του γράφου εισόδου.

4.1.2 calculate_n_cut_value

Η συνάρτηση αυτή χρησιμοποιείται μόνο κατά την εκτέλεση της **αναδρομικής** εκδοχής. Σκοπός της είναι ο υπολογισμός της μετρικής *Ncut*(*A,B*) για τις 2 ομάδες κόμβων. Δέχεται ως πίνακες (numpy arrays) εισόδου τους *affinity_mat*, που αντιπροσωπεύει τον γράφο κάποιας εικόνας και τον *cluster_idx*, μονοδιάστατος πίνακας μήκους *MN* με τις ετικέτες που δείχνουν σε ποιο από τα 2 cluster ανήκει ο κάθε κόμβος του γράφου. Αρχικά, γίνεται διαχωρισμός των κόμβων σε δύο ομάδες A και B με βάση τις τιμές του πίνακα *cluster_idx* μέσω της numpy.where (cluster_idx == 0 για την ομάδα A και cluster_idx == 1 για την ομάδα B). Στη συνέχεια υπολογίζεται το άθροισμα των βαρών εντός του κάθε cluster (*assoc_AA* και *assoc_BB*), καθώς και το άθροισμα των βαρών των ακμών που συνδέουν τους κόμβους κάθε cluster με όλους τους κόμβους του γράφου (*assoc_AV* και *assoc_BV*). Η διαδικασία αυτή εκτελείται με την βοήθεια των numpy.sum και numpy.ix_. Τέλος, διαιρούνται και προστίθενται οι όροι που υπολογίστηκαν σύμφωνα με τη θεωρία και η συνάρτηση επιστρέφει την ζητούμενη μετρική *n_cut (Ncut(A,B)).*

4.1.3 n_cuts_recursive

Η συνάρτηση αυτή χρησιμοποιείται επίσης μόνο κατά την εκτέλεση της **αναδρομικής** εκδοχής. Σκοπός της είναι ο έλεγχος επανεκτέλεσης διχοτόμησης μέσω των κριτηρίων thresholds *T*1, *T*2 και η διαδοχική διχοτόμηση του γράφου εισόδου χρησιμοποιώντας τις παραπάνω

συναρτήσεις. Δέχεται ως ορίσματα τον πίνακα (numpy array) *affinity_mat* και τα κατώφλια *T1, T2.* Αρχικά, εφαρμόζει τον αλγόριθμο Normalized Cuts (μέσω της συνάρτησης *n_cuts)* για δυαδική κατάτμηση (με k=2) και υπολογίζει τη σχετική τιμή N-cut (μέσω της συνάρτησης *calculate_n_cut_value).* Μέσω της συνθήκης *if,* ελέγχει αν ο αριθμός των κόμβων είναι μικρότερος ή ίσος του κατωφλίου T1, ή αν η τιμή *n_cut_value* είναι μεγαλύτερη από το όριο T2. Σε αυτές τις περιπτώσεις η συνάρτηση τερματίζει την αναδρομή και επιστρέφει την υπάρχουσα κατάτμηση *cluster_idx.* Διαφορετικά, εντοπίζει τους κόμβους κάθε υποομάδας με τη βοήθεια της numpy.where και για κάθε ομάδα κόμβων, φτιάχνει έναν νέο υποπίνακα συνεκτικότητας (*cluster_0_indices* και *cluster_1_indices)* όπου εφαρμόζει αναδρομικά την ίδια διαδικασία. Η ίδια συνάρτηση *n_cuts_recursive* καλείται ξανά για κάθε υποπίνακα και εξασφαλίζοντας ότι οι ετικέτες των ομάδων παραμένουν μοναδικές, τελικά συγχωνεύονται τα αποτελέσματα στον πίνακα *cluster_idx.* Με αυτόν τον τρόπο, η συνάρτηση δημιουργεί μια ιεραρχική ομαδοποίηση που σταματά είτε όταν τα υποσύνολα είναι πολύ μικρά είτε όταν η κατάτμηση θεωρείται κακής ποιότητας.

5 Demo

Οι απαιτούμενες ενέργειες εκτέλεσης της εργασίας, έχουν χωριστεί σε 5 διαφορετικά αρχεία ανάλογα με τη μέθοδο (συνάρτηση) που εκτελείται κάθε φορά. Για την εισαγωγή των προ-κατασκευασμένων βοηθητικών αρχείων γίνεται χρήση του πακέτου *scipy.io.loadmat* σύμφωνα με τις οδηγίας που έχουν υποδειχτεί, ενώ η απεικόνιση των αποτελεσμάτων γίνεται μέσω του *matplotlib.pyplot.*

5.1 *demo1.py*

Το αρχείο αυτό είναι υπεύθυνο για την παρουσίαση μόνο της λειτουργίας της ρουτίνας **Spectral Clustering**. Χρησιμοποιεί έναν προ-κατασκευασμένο affinity matrix *d1a* και καλεί τη συνάρτηση *spectral_clustering* για τις παραμέτρους *k = 2, k = 3* και *k = 4* (σύνολο 3 πειράματα). Τα αποτελέσματα που λαμβάνονται είναι τα εξής:

Clustering for k = 2

[1 1 1 1 0 0 0 0 0 0 0 0]

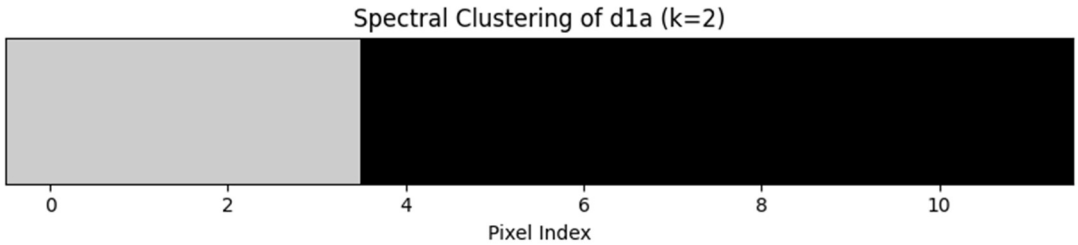
Clustering for k = 3

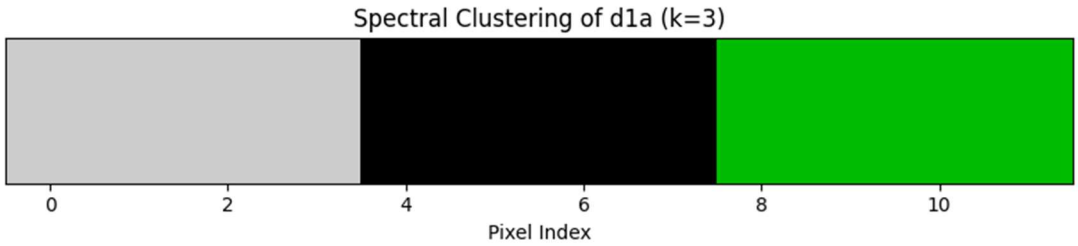
[2 2 2 2 0 0 0 0 1 1 1 1]

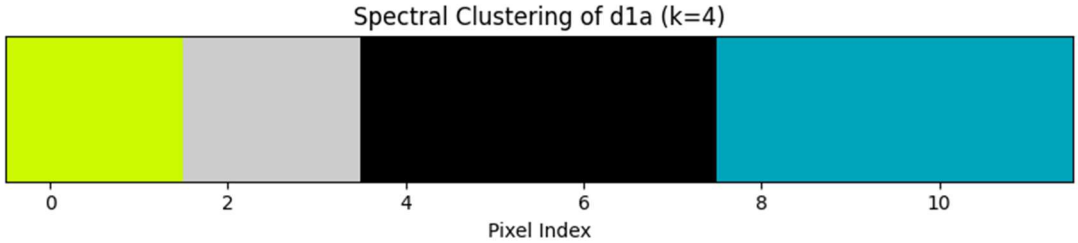
Clustering for k = 4

[2 2 3 3 0 0 0 0 1 1 1 1]

Επειδή δεν γνωρίζουμε το μέγεθος της αρχικής εικόνας επιλέχθηκε απεικόνιση των αποτελεσμάτων σε μορφή 1D color bars:



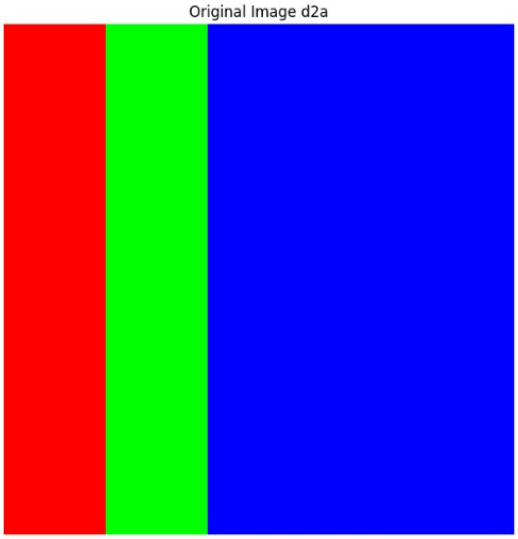




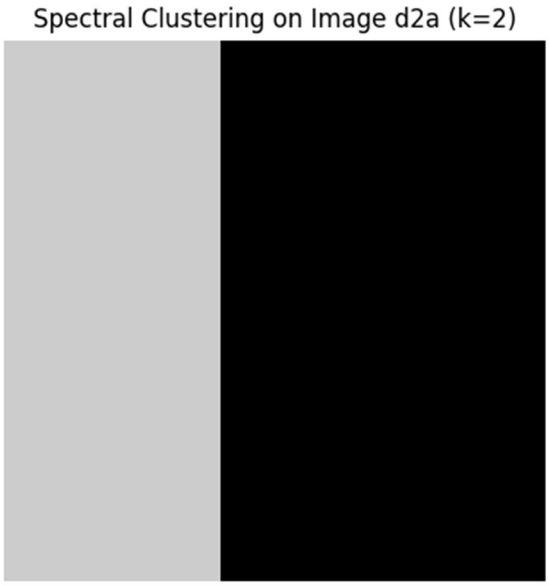
Αυτό που παρατηρούμε είναι ότι όλα τα clusters διατηρούν χωρική συνοχή, δηλαδή δεν υπάρχουν διακοπές ή κατακερματισμοί. Αυτό υποδηλώνει ότι η αρχική εικόνα είχε ομαλές μεταβάσεις. Τα αποτελέσματα δείχνουν ότι ο Spectral Clustering λειτουργεί αποδοτικά σε αυτή την απλή δομή, παράγοντας λογικούς και χωρικά συνεκτικούς διαχωρισμούς που εξελίσσονται προβλέψιμα με την αύξηση του k.

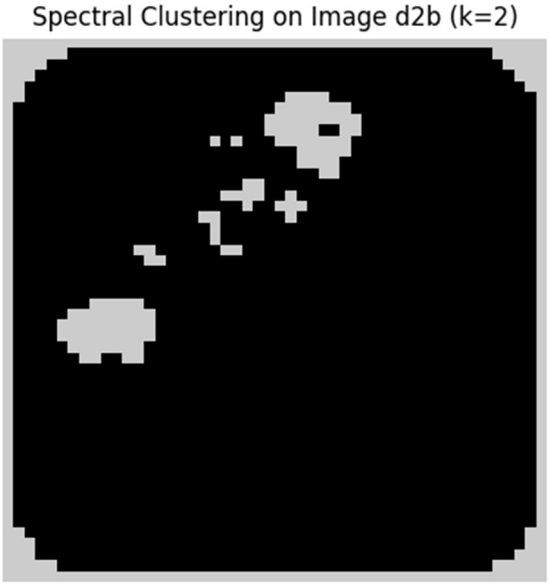
5.2 demo2.py

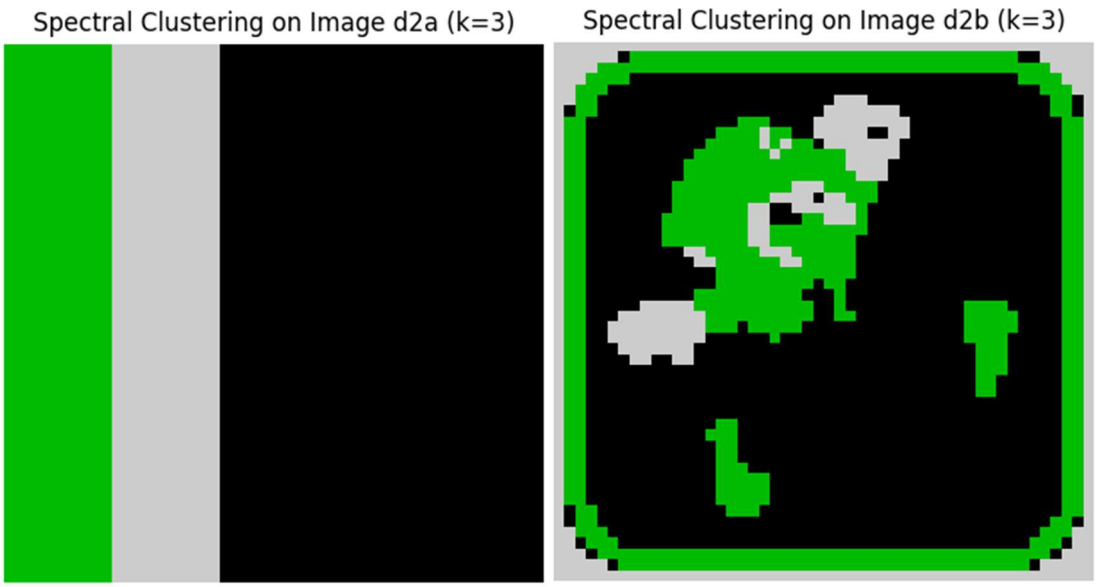
Το αρχείο αυτό είναι επίσης υπεύθυνο για την παρουσίαση της λειτουργίας της ρουτίνας **Spectral Clustering**, αλλά μέσω της διαδικασίας μετατροπής εικόνας σε "γράφο". Αρχικά οι δύο εικόνες εισόδου που θα χρησιμοποιηθούν τόσο σε αυτό το demo όσο και στα επόμενα είναι οι εξής:

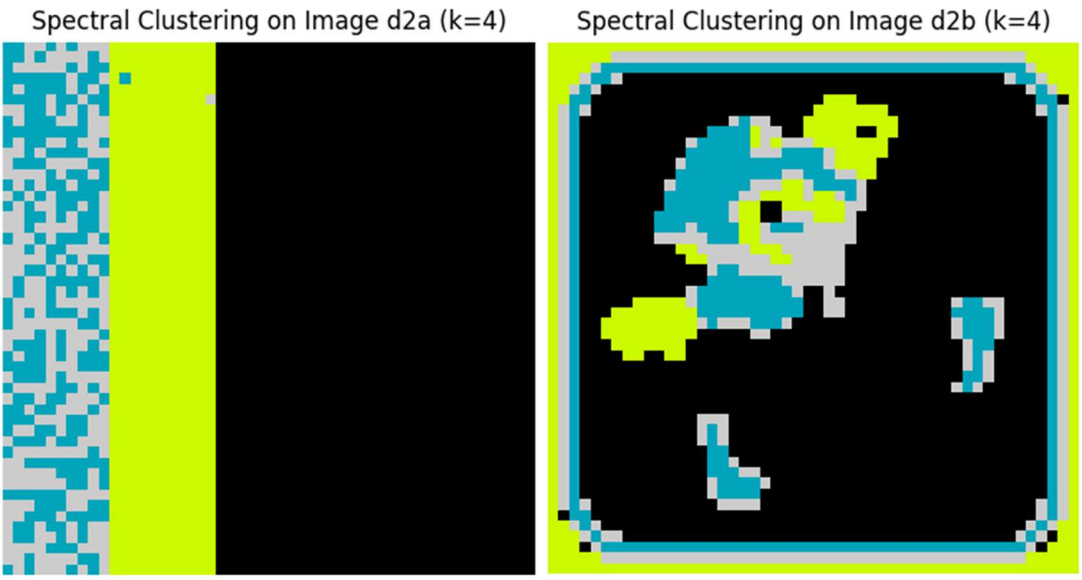


Πρόκειται για δύο προ-κατασκευασμένες RGB εικόνες εισόδου (σε μορφή numpy array) με ονόματα *d2a* και *d2b*, οι οποίες μετατρέπονται στον αντίστοιχο γράφο (δηλαδή στον αντίστοιχο affinity matrix) με την χρήση της *image_to_graph*. Έπειτα καλείται η συνάρτηση *spectral_clustering* για τις παραμέτρους $k = 2$, $k = 3$ και $k = 4$ και για τις δύο εικόνες (σύνολο 6 πειράματα) και ολοκληρώνεται η διαδικασία. Τα αποτελέσματα που λαμβάνονται είναι τα εξής:









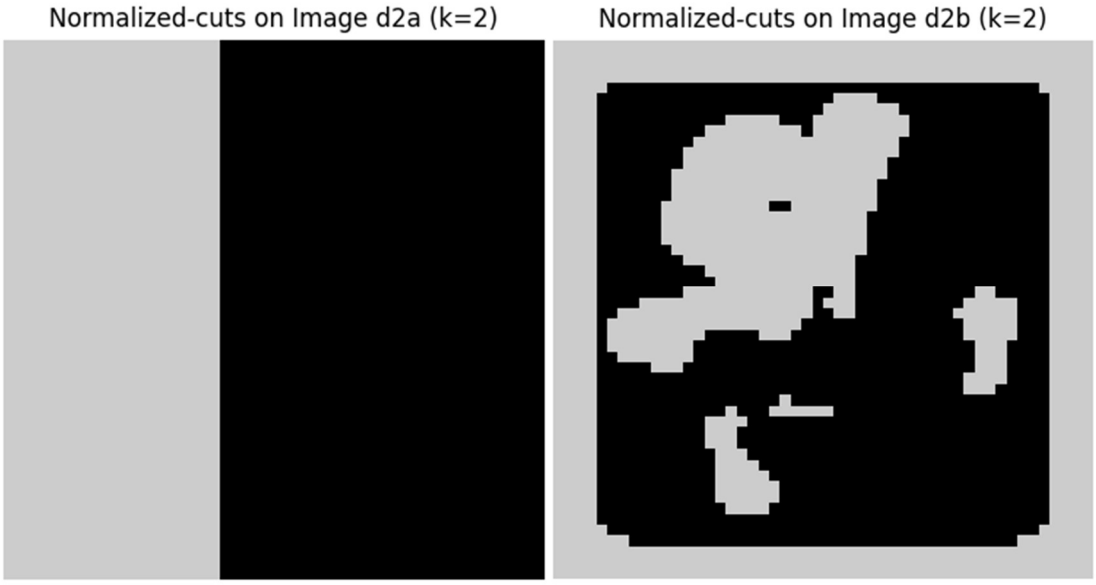
Εικόνα d2a: Φαίνεται να είναι μια απλή γεωμετρική δομή με κάθετες λωρίδες/περιοχές. Η δομή της είναι ομοιόμορφη.

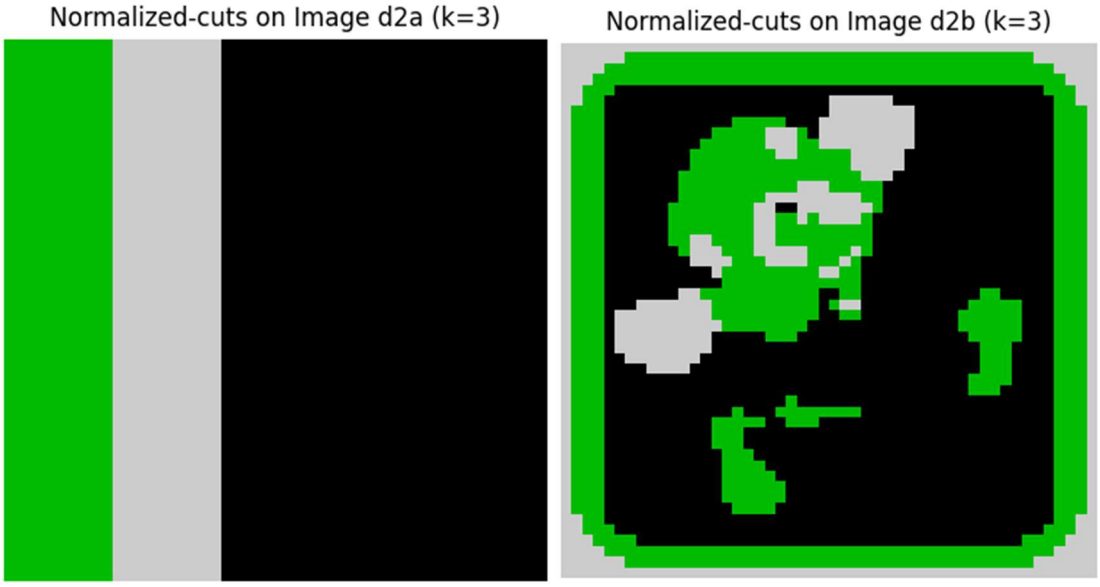
Εικόνα d2b: Έχει πιο σύνθετη δομή με διάσπαρτα αντικείμενα/περιοχές σε μαύρο φόντο. Περιλαμβάνει μικρότερες περιοχές που είναι χωρικά διαχωρισμένες.

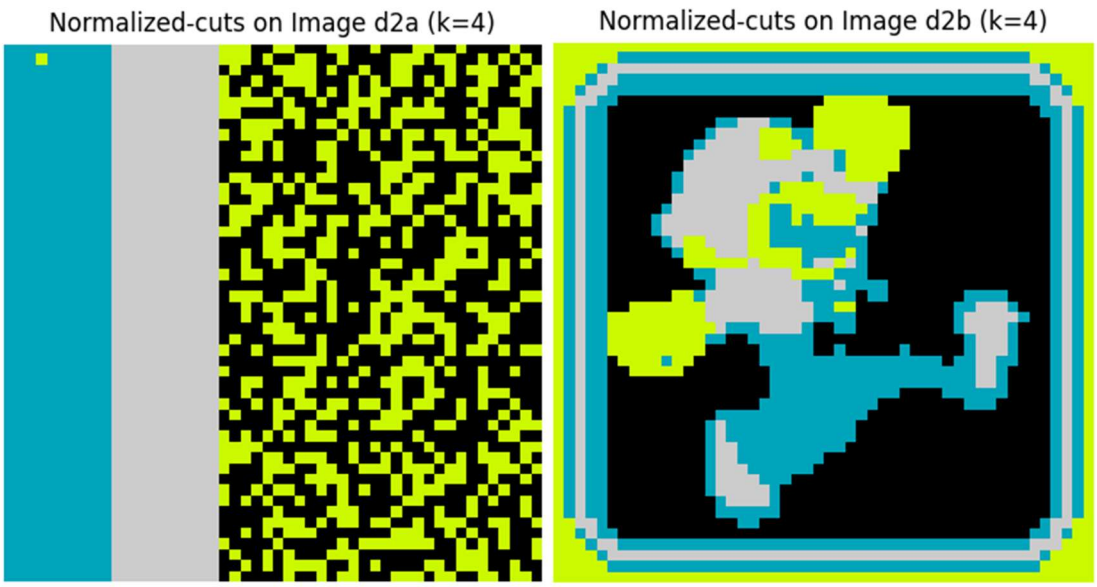
Παρατηρούμε ότι καθώς αυξάνει το k, εμφανίζονται νέα χρώματα που αντιπροσωπεύουν νέες υποπεριοχές, τις οποίες ο αλγόριθμος θεωρεί αρκετά διαφορετικές ώστε να σχηματίσουν ξεχωριστά clusters. Για k=4, η d2a αρχίζει να δείχνει υπερ-κατάτμηση (over-segmentation), ενώ η d2b φαίνεται να αναδεικνύει πραγματικά λεπτομέρειες της εικόνας.

5.3 demo3a.py

Το αρχείο αυτό αναλαμβάνει την εκτέλεση της μη-αναδρομικής ρουτίνας **Normalized-cuts.** Χρησιμοποιεί τις δύο προ-κατασκευασμένες RGB εικόνες εισόδου (σε μορφή numpy array) με ονόματα *d2a* και *d2b*, και τις μετατρέπει στον αντίστοιχο γράφο (δηλαδή στον αντίστοιχο affinity matrix) με την χρήση της *image_to_graph*. . Έπειτα καλεί τη συνάρτηση *n_cuts* για τις παραμέτρους *k = 2, k = 3* και *k = 4* και για τις δύο εικόνες (σύνολο 6 πειράματα) και ολοκληρώνει τη διαδικασία. Τα αποτελέσματα που λαμβάνονται είναι τα εξής:



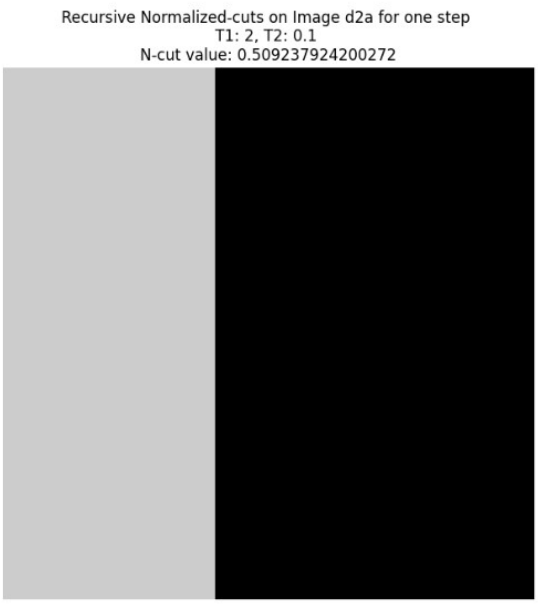


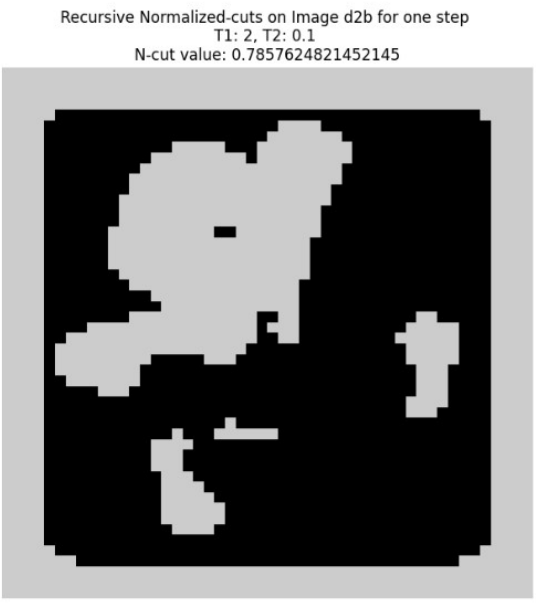


Παρατηρούμε ότι όπως και με τη μέθοδο Spectral Clustering, έτσι και με τη Normalized-Cuts, καθώς αυξάνει το k, εμφανίζονται νέα χρώματα που αντιπροσωπεύουν νέες υποπεριοχές, Η **Normalized Cuts** φαίνεται να είναι περισσότερο ευαίσθητη και να αναδεικνύει περισσότερες λεπτομέρειες μιας εικόνας, ενώ η **Spectral Clustering** τείνει να είναι πιο σταθερή και προβλέψιμη, ειδικά για απλές γεωμετρικές δομές.

5.4 demo 3b.py

Το αρχείο αυτό αναλαμβάνει την εκτέλεση της αναδρομικής ρουτίνας **Normalized-cuts** για ένα βήμα, δηλαδή σπάει τον αρχικό γράφο σε 2 μόνο κομμάτια**.** Χρησιμοποιεί τις δύο προ-κατασκευασμένες RGB εικόνες εισόδου (σε μορφή numpy array) με ονόματα *d2a* και *d2b*, και τις μετατρέπει στον αντίστοιχο γράφο (δηλαδή στον αντίστοιχο affinity matrix) με την χρήση της *image_to_graph*. Έπειτα καλείται η αναδρομική συνάρτηση *n_cuts_recursive* με τιμές κατωφλίων T1 και T2, τέτοιες ώστε να αποτρέπουν την αναδρομική εκτέλεση του αλγορίθμου για παραπάνω από ένα βήμα. Η διαδικασία εκτελείται παρομοίως και για τις δύο εικόνες (σύνολο 2 πειράματα) και τα αποτελέσματα που λαμβάνονται μαζί με τις τιμές κατωφλίων και τις τιμές Ncuct είναι τα εξής:

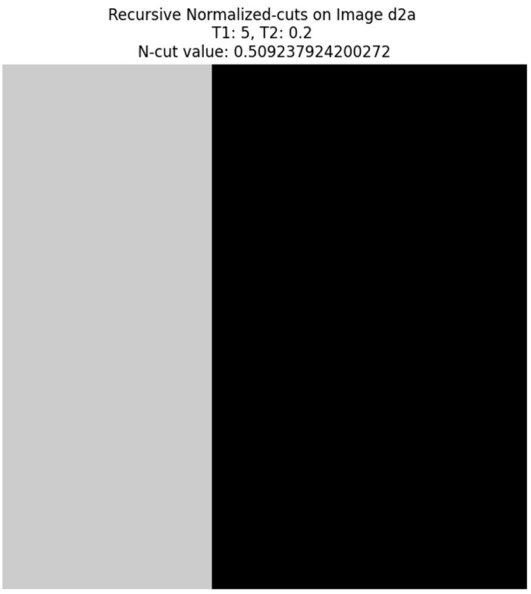


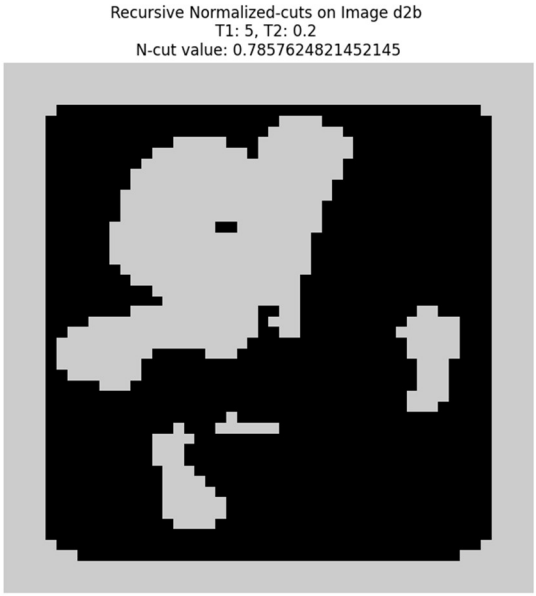


Σε σύγκριση με τη μέθοδο Spectral Clustering για k =2 παρατηρούμε ότι προφανώς και στις δύο περιπτώσεις γίνεται διχοτόμηση σε 2 clusters, ωστόσο η απεικόνιση είναι αρκετά καλύτερη με τη μέθοδο Normalized-cuts.

5.5 demo 3c.py

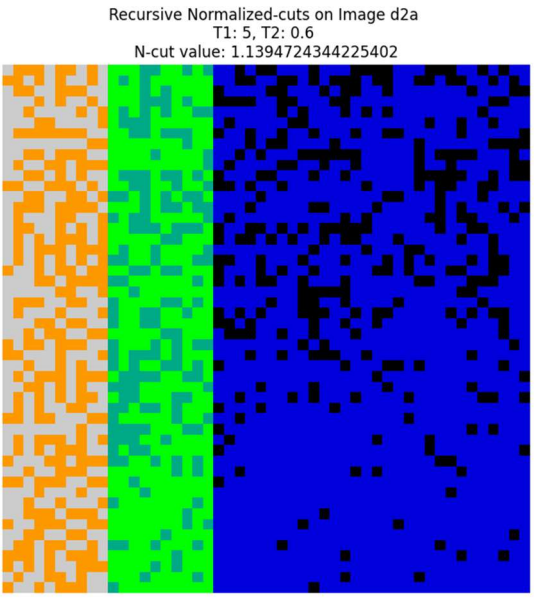
Το αρχείο αυτό αναλαμβάνει την εκτέλεση της ολοκληρωμένης αναδρομικής ρουτίνας **Normalized-cuts**, δηλαδή με την χρήση κατωφλίων. Αρχικά οι δύο προ-κατασκευασμένες RGB εικόνες εισόδου (σε μορφή numpy array) με ονόματα *d2a* και *d2b*, μετατρέπονται στον αντίστοιχο γράφο (δηλαδή στον αντίστοιχο affinity matrix) με την χρήση της *image_to_graph*. Έπειτα καλείται η αναδρομική συνάρτηση *n_cuts_recursive* με τιμές κατωφλίων T1 = 5 και T2 = 0.2, όπως δόθηκαν από την εκφώνηση. Η διαδικασία εκτελείται παρομοίως και για τις δύο εικόνες (σύνολο 2 πειράματα) και τα αποτελέσματα που λαμβάνονται είναι τα εξής:

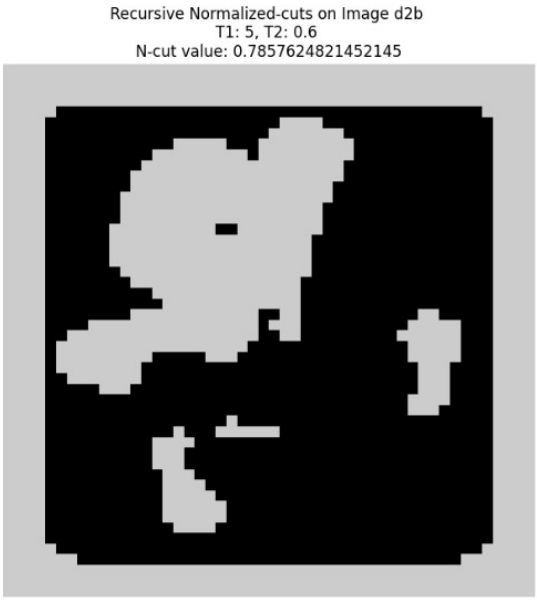




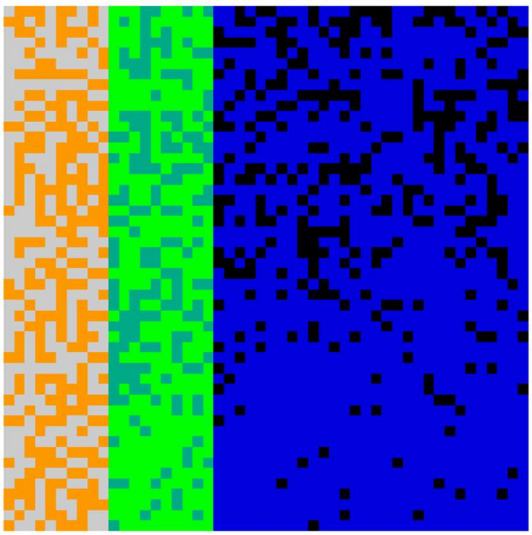
Από τα αποτελέσματα που λαμβάνονται συμπεραίνουμε ότι η αναδρομική διαδικασία εκτελέστηκε μόνο για ένα βήμα εφόσον προέκυψαν μόνο δύο clusters. Το αποτέλεσμα αυτό ήταν ζητούμενο και στο προηγούμενο demo3b. Συγκριτικά με την μέθοδο Spectral Clustering για k = 2 συμπεραίνουμε ότι και στις δύο περιπτώσεις έχουμε την ίδια διχοτόμηση σε 2 clusters, ωστόσο η αναδρομική Normalized-cuts προσφέρει πολύ καλύτερη απεικόνιση. Για k = 3 παρατηρούμε ότι η απεικόνιση είναι σχεδόν ίδια αλλά στη μέθοδο Spectral Clustering προκύπτει άλλο ένα καινούργιο χρώμα εκτός του μαύρου και του άσπρου, οπότε είναι προτιμότερη.

Προκειμένου να ελεγχθούν οι προοπτικές βελτίωσης μέσω της αναδρομικής μεθόδου, έγιναν δοκιμές με διαφορετικές τιμές κατωφλίων T1, T2 οι οποίες αναγράφονται παρακάτω:



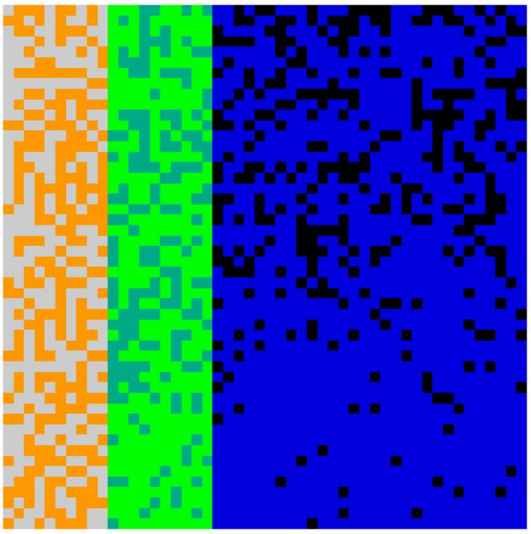


Recursive Normalized-cuts on Image d2a
T1: 5, T2: 0.8
N-cut value: 1.1394724344225402



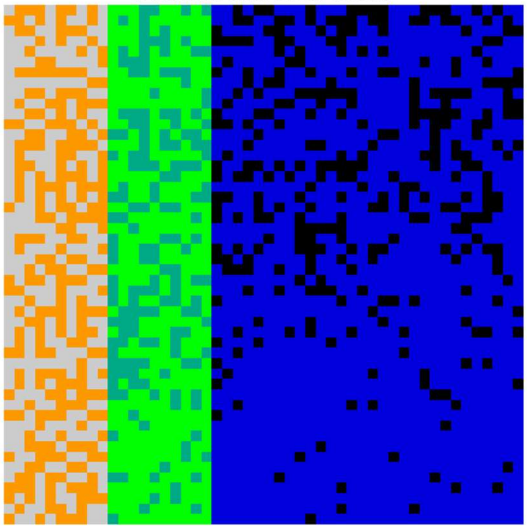
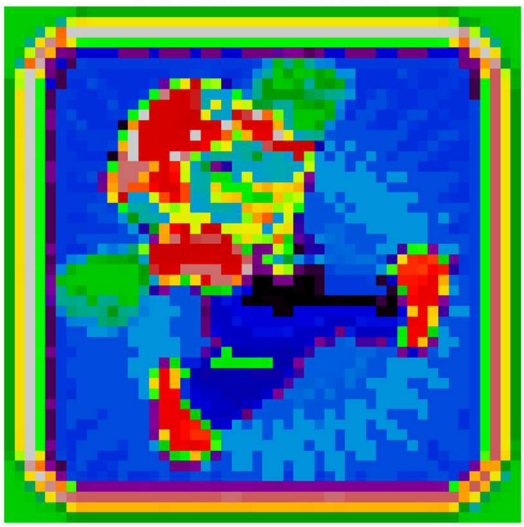
Recursive Normalized-cuts on Image d2b
T1: 5, T2: 0.8
N-cut value: 1.290572611741445

Recursive Normalized-cuts on Image d2a
T1: 5, T2: 0.95
N-cut value: 1.1394724344225402



Recursive Normalized-cuts on Image d2b
T1: 5, T2: 0.95
N-cut value: 1.9334492749212207

Recursive Normalized-cuts on Image d2a
T1: 5, T2: 0.99
N-cut value: 1.1394724344225402



Recursive Normalized-cuts on Image d2b
T1: 5, T2: 0.99
N-cut value: 1.9889078937702611